

Java List Interface

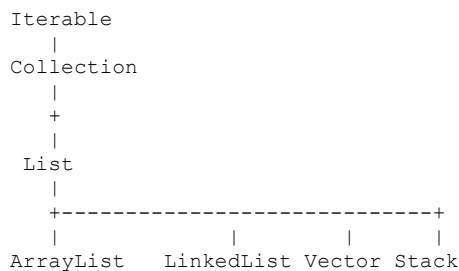
ArrayList • LinkedList • Vector • Stack

Data Structures using Java Collection Framework

Course Outcomes

- Understand List Interface
- Difference between ArrayList, LinkedList, Vector and Stack
- Internal Working
- Time Complexity
- Interview Questions
- Coding Problems
- Best Practices

Java Collection Framework Overview



Why Collection Framework?

- Dynamic Memory
- Reusable Data Structures
- Generic Programming
- Faster Development
- Standard APIs

What is List Interface?

A List is an ordered collection.

Properties

- ✓ Maintains insertion order
- ✓ Duplicate elements allowed
- ✓ Index based access
- ✓ Dynamic size
- ✓ Allows null values

Common Methods

```
add()
remove()
get()
set()
contains()
size()
clear()
indexOf()
lastIndexOf()
isEmpty()
```

List Interface Example

```
import java.util.*;

public class Demo {

    public static void main(String[] args) {

        List<String> names = new ArrayList<>();

        names.add("Amir");
```

```

        names.add("Rahul");
        names.add("Priya");

        System.out.println(names);

        System.out.println(names.get(1));

        names.remove(0);

        System.out.println(names);
    }
}

```

Output

```
[Amir, Rahul, Priya]
```

```
Rahul
```

```
[Rahul, Priya]
```

Slide 5

ArrayList

Internal Working

Index

```

0
1
2
3
4
5

```

```
-----
|A|B|C|D|E| |
-----
```

Stored in Continuous Memory

Characteristics

- ✓ Dynamic Array
- ✓ Random Access
- ✓ Fast Read
- ✓ Slow Middle Insertion

ArrayList Internal Working

When array becomes full

Before

```
[10] [20] [30] [40]
```

↓

Resize

↓

```
[10] [20] [30] [40] [ ] [ ] [ ] [ ]
```

Java internally creates a larger array and copies elements.

Time Complexity

Operation	Complexity
Add End	O(1) Amortized
Add Middle	O(n)
Remove Middle	O(n)
Search	O(n)
Get	O(1)

LinkedList

Internal Structure

NULL

↓

```
+-----+-----+
| Data | Next |
+-----+-----+
```

↓

```
+-----+-----+
| Data | Next |
+-----+-----+
```

↓

```
+-----+-----+
| Data | NULL |
+-----+-----+
```

Each node stores

- Data
- Address of next node

LinkedList Internal Working

Insertion

A → B → D

Insert C

A → B → C → D

Deletion

A → B → C → D

Delete C

A → B → D

Complexity

Operation	Complexity
Insert Beginning	O(1)
Insert End	O(1)
Insert Middle	O(n)
Delete	O(n)
Search	O(n)
Random Access	O(n)

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Storage	Array	Node
Memory	Less	More
Random Access	Fast	Slow
Insertion	Slow	Fast
Deletion	Slow	Fast
Search	O(n)	O(n)

Interview Question

Which is better?

Answer:

Depends upon use case.

Frequent insertion

➡ LinkedList

Frequent retrieval(random access)

➡ ArrayList

Vector

Old Collection Class

Introduced in JDK 1.0

Characteristics

- ✓ Dynamic Array
- ✓ Thread Safe
- ✓ Synchronized
- ✓ Slower than ArrayList

```
Vector<Integer> v = new Vector<>();
```

```
v.add(10);  
v.add(20);
```

Stack

LIFO

Push

```
30  
20  
10
```

↓

Pop

30 removed first

Operations

```
push()  
pop()  
peek()  
empty()  
search()
```

Example

```
Stack<Integer> st = new Stack<>();
```

```
st.push(10);  
st.push(20);  
st.push(30);  
System.out.println(st.pop());  
System.out.println(st.peek());
```

Stack vs ArrayDeque

Modern Java Recommendation

Instead of

```
Stack<Integer> st = new Stack<>();
```

Prefer

```
Deque<Integer> st = new ArrayDeque<>();
```

```
st.push(10);  
st.pop();  
st.peek();
```

Interview Question

Why?

Because Stack is legacy.

ArrayDeque is faster.

Iterating List

Method 1

`for(int x:list)`

Method 2

`Iterator`

Method 3

`ListIterator`

Method 4

`forEach()`

Method 5

`Stream API`

Interview Questions

- ✓ Difference between ArrayList and LinkedList?
- ✓ Why ArrayList is faster?
- ✓ Why LinkedList consumes more memory?
- ✓ ArrayList resizing?
- ✓ Initial capacity?
- ✓ Capacity growth?
- ✓ Difference between Vector and ArrayList?
- ✓ Difference between Stack and Queue?
- ✓ Difference between Stack and ArrayDeque?
- ✓ Is List synchronized?
- ✓ Can List store duplicates?
- ✓ Can List store null?
- ✓ Fail Fast vs Fail Safe Iterator?
- ✓ ConcurrentModificationException?

Real-world Applications

ArrayList

- Student Management
- Employee Records
- Product Catalog
- Shopping Cart

LinkedList

- Music Playlist
- Browser History
- Undo/Redo
- Navigation

Stack

- Function Calls
- Undo
- Expression Evaluation
- DFS
- Backtracking

Vector

- Legacy Enterprise Applications

Summary

Implementation	Best Use Case
ArrayList	Fast Retrieval
LinkedList	Frequent Insert/Delete
Vector	Thread-safe Legacy Code
Stack	LIFO Operations
ArrayDeque	Modern Stack Implementation

Live Coding Demonstrations

During the lecture, consider these live coding exercises:

1. CRUD operations on `ArrayList`
2. Convert `ArrayList` to `LinkedList`
3. Sort using `Collections.sort()`
4. Iterate using `Iterator`, `ListIterator`, enhanced `for`, and `Streams`
5. Demonstrate `ConcurrentModificationException`
6. Show `ArrayList` resizing concept
7. Implement a custom stack using `ArrayList`
8. Compare execution time of `ArrayList` VS `LinkedList` for insertion and retrieval
9. Demonstrate `Stack` VS `ArrayDeque`
10. Use `Collections.reverse()`, `shuffle()`, `binarySearch()`, and `frequency()`

Interview Cheat Sheet

Question	Expected Answer
Why is <code>ArrayList.get()</code> $O(1)$?	Backed by a contiguous array with direct index access.
Why is insertion in the middle of <code>ArrayList</code> $O(n)$?	Elements after the insertion point must be shifted.
Why does <code>LinkedList</code> use more memory?	Each node stores data plus next/previous references.
Why is <code>Vector</code> slower than <code>ArrayList</code> ?	Methods are synchronized, adding overhead.
Why is <code>Stack</code> considered legacy?	<code>ArrayDeque</code> provides better performance and is the recommended LIFO implementation.
Which list should be used most of the time?	<code>ArrayList</code> , unless frequent insertions/deletions in the middle justify <code>LinkedList</code> .

Practice Problems (Active Links)

ArrayList

Problem	Link
ArrayList in Java Practice	GeeksforGeeks – ArrayList in Java
ArrayList Operations	GeeksforGeeks – Java ArrayList Methods
Find Pair with Given Sum	GeeksforGeeks – Pair with Given Sum
Union of Two Arrays	GeeksforGeeks – Union of Two Arrays
Leaders in an Array	GeeksforGeeks – Leaders in an Array

LinkedList

- Reverse Linked List — <https://leetcode.com/problems/reverse-linked-list/>
- Middle of the Linked List — <https://leetcode.com/problems/middle-of-the-linked-list/>
- Linked List Cycle — <https://leetcode.com/problems/linked-list-cycle/>
- Merge Two Sorted Lists — <https://leetcode.com/problems/merge-two-sorted-lists/>
- Remove Nth Node From End of List — <https://leetcode.com/problems/remove-nth-node-from-end-of-list/>

Stack

- Valid Parentheses — <https://leetcode.com/problems/valid-parentheses/>
- Min Stack — <https://leetcode.com/problems/min-stack/>
- Evaluate Reverse Polish Notation — <https://leetcode.com/problems/evaluate-reverse-polish-notation/>
- Daily Temperatures — <https://leetcode.com/problems/daily-temperatures/>